

AIKernel Semantic DSL Compiler and Deterministic Agent Execution Architecture

Compiling AI-Generated Plans into Governed Pipelines with Semantic IR, ReplayLogs, and Fail-Closed Control

Author: Takuya Sogawa

Affiliation: AIKernel Project

ORCID: <https://orcid.org/0009-0009-7499-2595>

DOI: <https://doi.org/10.5281/zenodo.20534341>

Version: v0.1.0-rev1

Date: 2026-06-04

Status: Technical Note / Architecture Draft

License: CC BY 4.0

The English manuscript is the canonical version. The Japanese manuscript is included as a companion translation.

Abstract

This technical note introduces the **AIKernel Semantic DSL Compiler**, an OS-level mechanism for governing probabilistic LLM-generated plans through deterministic compilation rather than direct execution. The core novelty is a runtime boundary where the model generates a structured intent, while AIKernel validates, normalizes, compiles, executes, and records that intent as a governed artifact.

The compilation path is: **Semantic IR -> Admissibility Checking -> Governed Pipeline -> ResultStep / SemanticDelta -> hash-linked ReplayLog**. In this model, the DSL is not executable source code and is not Turing-complete. It is an intent-level intermediate representation that the kernel may admit, reject, suspend, clarify, or compile under fail-closed governance.

The architecture addresses a common weakness of autonomous agent frameworks: planning and execution often remain entangled inside a single nondeterministic loop. AIKernel separates those responsibilities. The LLM proposes; the Semantic Compiler resolves capabilities, checks policies, verifies bounded control flow, and constructs a deterministic pipeline; the executor applies only explicit semantic deltas and records every step in a replayable hash chain.

The note positions the Semantic DSL Compiler as an AIKernel Phase-2.5 mechanism, bridging Phase-2 semantic compilation theory and Phase-3 just-in-time semantic execution. It does not claim that LLM planning becomes deterministic. Instead, it defines how nondeterministic proposals can be transformed into deterministic, auditable, and fail-closed execution artifacts.

Keywords

AIKernel; Semantic DSL; Semantic IR; Deterministic Agent Execution; Governed Pipeline; ReplayLog; Fail-Closed Governance; Capability Resolution; Semantic Compiler; C#; LINQ; Result Monad; Autonomous AI Agents; Phase-2.5

1. Introduction

LLM-based agents can convert natural-language instructions into complex multi-step task plans. They can call tools, retrieve files, browse repositories, draft documents, generate code, and coordinate workflows. This expressive power creates a new software-engineering problem: the agent's reasoning and the system's execution boundary are often collapsed into one nondeterministic loop.

In many existing agent frameworks, the model proposes an action and the runtime executes it with only local validation. The execution path may be observable as logs, but the semantic state transition itself is rarely represented as a deterministic, replayable, policy-checked artifact. This makes it difficult to answer operational questions such as: Why was this tool selected? Which capability authorized it? Which policy admitted the step? What state changed? Could the same state be reconstructed from the recorded trace?

AIKernel treats the LLM as a stochastic process running above a deterministic semantic operating layer. The model may generate text, plans, candidates, or explanations; however, it must not directly mutate privileged system state. Between stochastic planning and execution, AIKernel introduces a kernel boundary: a **Semantic DSL Compiler**.

The central idea is simple: an AI-generated plan should be treated as **data, not code**. The DSL is a structured intent declaration. It is parsed into an abstract syntax tree (AST), validated against available capabilities and governance rules, normalized into a constrained semantic intermediate representation, and compiled into a **Governed Pipeline**. Only the compiled pipeline may be executed.

This design separates three concerns:

1. **Planning:** the LLM proposes a structured plan under uncertainty.
2. **Compilation:** AIKernel deterministically validates and reconstructs the plan.
3. **Execution:** the governed pipeline is evaluated through explicit result and delta semantics.

The result is not a general-purpose programming language for agents. It is a restricted semantic execution contract for transforming probabilistic proposals into replayable, auditable, fail-closed runtime transitions.

2. Phase-2.5 Positioning in AIKernel

This note is positioned as **AIKernel Phase-2.5**. It bridges the semantic compilation concepts of Phase-2 and the future Phase-3 runtime model in which semantic compilation may be performed just-in-time against live capabilities, VFS state, ROM identity, and execution policy.

The relationship can be summarized as follows:

Phase-1: Capability, VFS, ROM, ReplayLog, Governance
Phase-2: Semantic IR and semantic compilation theory
Phase-2.5: Semantic DSL Compiler for governed agents
Phase-3: JIT Semantic Compiler and runtime semantic OS

The Semantic DSL Compiler depends on Phase-1 concepts such as CapabilityROM, VFS, ReplayLog, and fail-closed policy enforcement. It also prepares the ground for Phase-3 by defining how generated intent can be compiled into deterministic execution artifacts instead of being executed as untrusted code.

In this sense, Phase-2.5 is the bridge between **semantic representation** and **deterministic agent execution**.

3. Background: AIKernel Semantic Runtime

AIKernel is a Semantic Context OS architecture for governing LLM-centered systems. Its purpose is not to replace language models, but to define the deterministic boundary around them.

3.1 Provider and Capability

A **Provider** is an implementation of an external or internal capability: a model endpoint, file system access path, repository interface, document converter, search adapter, or domain-specific operator. A **Capability** describes what can be used, under which policy, and with what operational risk.

The Semantic DSL Compiler does not allow arbitrary tool names to become executable calls. It resolves every requested operation against a registered capability table. Unknown providers, missing capabilities, ambiguous aliases, or policy-denied operations produce compilation failure.

3.2 ReplayLog

A **ReplayLog** is a deterministic trace record. In AIKernel, it records input state, selected providers, normalized parameters, execution outputs, governance decisions, and resulting semantic deltas. Replay entries are hash-linked so that historical modification is detectable.

For agent execution, replayability is more than debugging. It is the mechanism by which a probabilistic interaction is converted into a deterministic audit object.

3.3 Fail-Closed Governance

Fail-closed governance means that undefined, ambiguous, malformed, unauthorized, or indeterminate states do not proceed by default. In a conventional program, an unexpected exception may propagate unpredictably. In AIKernel, such states are represented as governed failures and recorded explicitly.

The Semantic DSL Compiler extends fail-closed governance from single prompt transactions to multi-step agent pipelines.

4. Design Goals and Non-Goals

The architecture has four design goals.

First, it separates **plan generation** from **execution authorization**. The LLM may propose a pipeline, but the kernel determines whether the proposal is admissible.

Second, it converts untrusted generated structure into a deterministic **Semantic IR**. The compiler is responsible for parsing, normalization, validation, capability resolution, and policy checks.

Third, it makes every execution step replayable. Each step produces an explicit ResultStep, a SemanticDelta, and a hash-linked ReplayLog record.

Fourth, it provides bounded control flow for autonomous agents. Loops, suspend/resume points, human approval gates, and external-event waits must be represented as governed nodes with explicit limits.

The model does not claim that LLM reasoning is deterministic. It also does not claim to solve all prompt injection or tool misuse problems by itself. Sandboxing, permissioning, capability scoping, policy evaluation, and provider isolation remain necessary. The contribution is the deterministic compilation and execution boundary.

5. Semantic DSL as Structured Semantic IR

The DSL is not C#, Python, JavaScript, shell script, or any other Turing-complete source language. It is a constrained Semantic IR that describes intent-level execution steps. The AI generates a **structured intent**, not an executable program.

5.1 The Four Slots of Semantic IR

For alignment with AIKernel Phase-2 terminology, a Semantic IR element can be represented as:

$x = (G, T, C, B)$

G: Goal slot	- intended outcome or root objective
T: Tool slot	- requested capability or provider family
C: Context slot	- required inputs, VFS references, ROM facts
B: Boundary slot	- policy, risk, loop, approval constraints

The important point is that this tuple is descriptive. It does not execute. It becomes executable only after the kernel compiler proves that the requested capability, context, and boundary conditions are admissible.

5.2 Pipeline Root

A valid document has a single root node of type Pipeline. A pipeline contains bounded steps, declared dependencies, requested capabilities, input bindings, output bindings, and governance metadata.

A minimal conceptual form is shown below.

```
{
  "type": "Pipeline",
  "pipelineId": "example-log-review",
  "goal": "Summarize yesterday's application errors",
  "steps": [
    {
      "id": "s1",
      "type": "CapabilityCall",
      "capability": "vfs.readText",
      "input": { "path": "logs/app-2026-06-03.log" },
      "policy": { "mode": "readOnly" }
    },
    {
      "id": "s2",
      "type": "ModelTransform",
      "dependsOn": ["s1"],
      "capability": "model.summarize",
      "input": { "fromStep": "s1" }
    }
  ]
}
```

This structure is executable only after compilation. Before compilation, it is merely a proposal.

5.3 Intent Declaration, Not Arbitrary Code

The DSL is intentionally declarative. It does not expose arbitrary loops, eval, reflection, dynamic imports, shell expansion, unrestricted file paths, or open-ended tool execution. Every operation must correspond to a known node type and a registered capability.

This distinction reduces the attack surface. A malicious prompt may induce the model to generate a dangerous plan, but the plan still passes through deterministic schema validation, capability resolution, policy checks, and risk evaluation before execution.

6. Semantic Compiler Architecture

The Semantic Compiler transforms a proposed DSL document into an executable Governed Pipeline. The compilation path has four stages.

6.1 Parsing

The parser maps the JSON or AST representation into internal PipelineNode structures. Parsing is purely syntactic. It validates object shape, required fields, type tags, and basic identifiers.

6.2 Normalization

Normalization canonicalizes identifiers, orders dependencies, resolves aliases, expands defaults, and produces a deterministic representation. Two equivalent plans should produce the same normalized IR when evaluated under the same compiler version and policy context.

6.3 Formal Admissibility Checking

The compiler then performs semantic admissibility checks. Let x denote a normalized Semantic IR element. AIKernel admits x only if the following predicate holds:

```
A(x) =  
  schema_ok(x)  
  AND capability_ok(x)  
  AND policy_ok(x)  
  AND dataflow_ok(x)  
  AND loop_bounded(x)  
  AND resource_bounded(x)  
  AND provider_bound(x)  
  AND canonicalizable(x)  
  AND NOT indeterminate(x)
```

The acceptance rule is:

```
Accept(x) iff A(x) = true and PDP(x) = Permit  
Reject(x) otherwise
```

The admissibility result may be:

- Admit;
- Deny;
- SuspendForApproval;
- Clarify;
- Indeterminate.

Indeterminate is treated as deny at the execution boundary. This preserves the fail-closed contract.

Fail-closed rejection is triggered when any of the following conditions are present:

- unknown node type;
- missing required field;
- duplicate step identifier;
- unresolved dependency;
- cycle in a DAG-required region;
- missing capability;
- policy-denied capability;
- loop without maxIterations or timeout;
- external wait without a resume condition;
- ambiguous policy state;
- provider identity cannot be canonicalized;
- replay-hash input cannot be serialized deterministically.

6.4 Pipeline Construction

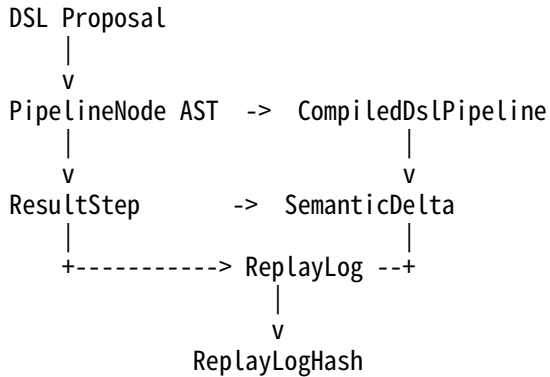
After all checks pass, the compiler constructs a Governed Pipeline. A pipeline may be a DAG for acyclic regions and may contain bounded control nodes for loop-like behavior. Even when a loop node exists, it must be represented as a bounded state machine rather than an unconstrained runtime loop.

7. Deterministic Execution Model

A compiled pipeline is executed through deterministic step evaluation. Each step consumes a context snapshot and returns a `ResultStep`.

7.1 Execution Trace Overview

The following diagram shows the relationship among the main runtime artifacts:



The model is intentionally staged. DSL parsing, compilation, execution, delta application, and replay hashing are different responsibilities.

7.2 ResultStep and SemanticDelta

A `ResultStep` captures the result of a single governed step. It includes the step identifier, parent identifiers, normalized input hash, provider identity, policy decision, output hash, and a `SemanticDelta`.

A `SemanticDelta` is not an uncontrolled side effect. It is a structured description of intended state change. The executor applies deltas only through kernel-governed transition rules.

Conceptually:

`Context_t + ResultStep_t + SemanticDelta_t -> Context_{t+1}`

This makes state transition explicit and auditable.

7.3 Hash-Linked ReplayLog

Each step appends a replay entry. The entry includes the previous replay hash, creating a deterministic hash chain.

`ReplayHash_t = H(ReplayHash_{t-1} || CanonicalReplayEntry_t)`

The hash chain binds plan, context, capability choice, policy decision, step output, and resulting semantic delta. Any later modification changes the replay hash and breaks trace verification.

7.4 Result Monad and LINQ Bind

In C#/.NET, the execution model maps naturally to `Task<Result<T>>` and LINQ `SelectMany` composition. Rather than throwing exceptions for expected governance outcomes, each stage returns a `Result`.

```
Task<Result<PipelineContext>> executed =
    from ast in parser.ParseAsync(document)
    from compiled in compiler.CompileAsync(ast, policyContext)
    from result in executor.ExecuteAsync(compiled)
    select result;
```

The purpose of this style is not syntactic elegance alone. It makes failure propagation explicit. A denied capability, malformed node, or indeterminate policy decision becomes `Result.Failure`, preserving deterministic control flow and replayability.

8. Agent Control Flow in DSL

Autonomous agents require more than linear pipelines. They need loops, approvals, waits, retries, suspensions, and recovery. The Semantic DSL supports these only as governed control nodes.

8.1 Bounded Loops

`Loop` and `LoopUntil` nodes must specify `maxIterations`, `timeout`, and exit condition. The compiler rejects unbounded loops.

A loop is represented as a bounded state transition region, not an arbitrary while-loop controlled by generated code. This prevents resource exhaustion and makes replay finite. Each iteration is recorded as a replayable step:

```
{
  "stepId": "loop.check-errors#3",
  "loopId": "loop.check-errors",
  "iteration": 3,
  "maxIterations": 5,
  "exitConditionHash": "...",
  "previousReplayHash": "..."
}
```

The iteration counter is part of the hash input. Therefore, loop replay is bounded and causally visible.

8.2 Suspend and Resume

Some operations require human approval or external events. In such cases, the pipeline emits a `Suspend` state. `Suspend` is neither success nor failure; it is a governed interruption point with no semantic delta applied.

The replay log records the suspension point, required approval, resume condition, and current replay hash. On resume, `AIKernel` restores the prior context from the replay log, verifies the hash chain, checks whether the approval or event satisfies the resume contract, and continues execution from the recorded node.

A resume operation must satisfy two conditions:

1. the prior suspension replay hash must match the stored trace; and
2. the approval or event must satisfy the declared resume contract.

If either condition is indeterminate, the pipeline remains suspended or fails closed.

8.3 Retry and Recovery

Retries must be declared as bounded recovery policies. A retry without a limit is invalid. A recovery branch must name the failed step, the admissible error class, and the maximum retry count.

This allows dynamic behavior without abandoning deterministic governance.

9. Governance and Safety

The Semantic DSL Compiler applies governance at three layers.

9.1 Compile-Time Governance

Compile-time governance rejects invalid structures before any provider is invoked. It checks schema, capabilities, dependencies, policy, loops, and resource bounds.

9.2 Step-Time Governance

Before each step executes, the provider binding and context are rechecked. This prevents time-of-check/time-of-use drift, especially when long-running pipelines resume after external approval.

9.3 Post-Step Governance

After a step completes, the resulting `SemanticDelta` is checked before it is applied. If the delta exceeds policy, violates capability scope, or conflicts with the root goal, the pipeline stops fail-closed.

This three-layer approach makes governance part of the execution semantics rather than an external afterthought.

10. Implementation Profile in AIKernel.NET

A reference implementation can be structured around the following interfaces.

```
public interface IDslParser
{
    Task<Result<PipelineNode>> ParseAsync(string document);
}
```

```
public interface ISemanticCompiler
{
    Task<Result<GovernedPipeline>> CompileAsync(
        PipelineNode node,
        PolicyContext policyContext);
}
```

```
public interface IPipelineExecutor
{
    Task<Result<PipelineExecutionResult>> ExecuteAsync(
        GovernedPipeline pipeline,
        KernelExecutionContext context);
}
```

```
public interface IKernelReplayer
{
    Task<Result<KernelExecutionContext>> RestoreAsync(
        ReplayTrace trace);
}
```

The implementation should follow AIKernel performance discipline: avoid unnecessary materialization on hot paths, use deterministic canonicalization where replay hashes depend on byte-level stability, and treat provider output as untrusted until validated.

The implementation should also preserve a strict boundary between generated DSL and executable code. The compiler may interpret a node type, but it must not evaluate generated code.

11. Evaluation and Validation Matrix

This note is architectural and does not claim throughput benchmarks. The following validation matrix defines the expected experimental acceptance tests for a reference prototype.

Test	Procedure	Expected result
Replay determinism	Same DSL, same inputs, same policy, same provider outputs	Identical final <code>ReplayLogHash</code>
ROM tamper rejection	Modify a <code>CapabilityROM</code> or <code>VFS</code> identity hash before compile	Compilation rejects the pipeline
Capability denial	Request a capability outside policy scope	Result: Failure with no delta applied
Suspend/resume continuity	Suspend for approval, then resume from recorded hash	Resume only if prior hash matches
Bounded loop check	Run <code>LoopUntil</code> with <code>maxIterations = 5</code>	At most five iteration records
Indeterminate PDP	Force policy lookup timeout or ambiguous decision	Fail-closed rejection
Canonicalization stability	Reorder equivalent fields in DSL input	Same normalized IR hash

The central experimental claim is not that every task succeeds. It is that every admitted execution path is governed, replayable, and reproducible under the same inputs, policy context, provider identities, and recorded provider outputs.

12. Related Work

Existing systems provide important pieces of the agent execution problem, but they typically do not combine deterministic semantic compilation, capability-governed admissibility, and cryptographic replay traces.

System	Relevant strength	Difference from AIKernel
LangChain / LangGraph	Agent orchestration and graph workflows	Logs exist, but semantic deltas and replay hashes are not the kernel boundary
AutoGen	Multi-agent conversation and task coordination	Execution semantics are conversational and less deterministic
Semantic Kernel	Plugins, planners, and process abstractions	It does not define a non-Turing-complete Semantic DSL compiler with <code>ReplayLog</code> hash chains
WASM runtimes	Sandboxed deterministic bytecode execution	They execute code, not semantic intent; governance is not semantic by default
Workflow engines	Explicit process graphs	They usually lack LLM-origin admissibility checks and AIKernel-style capability governance

AIKernel differs in emphasis. It treats the agent plan as an untrusted semantic artifact that must be compiled through a deterministic kernel boundary before execution. The primary object is not only a workflow graph, but a replayable state-transition contract with hash-linked evidence, explicit capability resolution, and fail-closed governance.

The architecture also relates to functional error handling, monadic composition, workflow engines, and policy-based access control. Its distinctive contribution is the combination of stochastic plan generation, Semantic IR compilation, deterministic execution, and cryptographic replay trace formation in a single kernel-level agent architecture.

13. Limitations

This model has four main limitations.

13.1 Expressiveness Limits

The DSL is intentionally not Turing-complete. This improves safety but limits expressiveness. Some legitimate workflows may require decomposition into smaller governed nodes or explicit provider-side implementations.

13.2 Dependence on Provider Determinism

Deterministic replay depends on deterministic provider behavior or recorded provider outputs. External APIs, time-dependent systems, and nondeterministic model calls must be isolated, recorded, mocked, or replay-substituted.

13.3 Hash-Chain Stability

Replay hash stability depends on stable canonicalization. Serialization order, provider identity, policy version, timestamps, file identities, and context snapshots must be recorded in a deterministic byte representation.

13.4 Intent Extraction Uncertainty

The compiler can reject or admit a structured intent, but it cannot guarantee that the LLM extracted the user's true intent perfectly. Clarification, user confirmation, and root-goal governance remain necessary.

14. Conclusion

This technical note proposed the AIKernel Semantic DSL Compiler as a deterministic execution boundary for autonomous agents. The model treats AI-generated plans as structured Semantic IR, not executable code. It compiles those plans into governed pipelines after capability resolution, policy checking, control-flow bounding, and fail-closed validation.

Execution proceeds through explicit `ResultStep` and `SemanticDelta` values, with replay entries linked into a deterministic hash chain. This converts stochastic agent proposals into auditable runtime artifacts while preserving the principle that the LLM proposes and the kernel governs.

As a Phase-2.5 mechanism, the Semantic DSL Compiler connects AIKernel's existing VFS, CapabilityROM, ReplayLog, and governance layers to a future Phase-3 JIT Semantic Compiler. Future work includes formal schema definitions, replay-hash canonicalization tests, recovery-branch semantics, distributed pipeline execution, and integration with the broader AIKernel governance layer.

References

- Microsoft. (2026). Semantic Kernel documentation.
<https://learn.microsoft.com/en-us/semantic-kernel/>
- Microsoft. (2026). Microsoft Semantic Kernel repository.
<https://github.com/microsoft/semantic-kernel>
- LangChain. (2026). LangChain documentation.
<https://docs.langchain.com/>
- LangChain. (2026). LangGraph repository: Build resilient agents.
<https://github.com/langchain-ai/langgraph>

- Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, S., Zhu, E., Jiang, B., Zhang, L., Zhang, S., Liu, J., Awadallah, A. H., White, R. W., Burger, D., & Wang, C. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. OpenReview.
<https://openreview.net/forum?id=BAakY1hNKS>
- Moggi, E. (1991). Notions of computation and monads. *Information and Computation*, 93(1), 55-92.
[https://doi.org/10.1016/0890-5401\(91\)90052-4](https://doi.org/10.1016/0890-5401(91)90052-4)
- Wadler, P. (1995). Monads for functional programming. *Advanced Functional Programming*, 24-52.
https://doi.org/10.1007/3-540-59451-5_2
- Sogawa, T. (2026). Interface-Led Architecture (ILA): A Software Development Methodology for the AI Era, Validated by the AIKernel Execution Model. Zenodo.
<https://doi.org/10.5281/zenodo.20290614>
- Sogawa, T. (2026). AIKernel Formal Foundations: Contract-Based Semantic Execution for Governed AI Systems. Zenodo.
<https://doi.org/10.5281/zenodo.20460322>
- Sogawa, T. (2026). AIKernel Hash-Anchored Trust Layer (HATL): A Hybrid Symmetric Ledger with Hash-Based Public Anchors. Zenodo.
<https://doi.org/10.5281/zenodo.20502685>